Interview Q&A: Distributed Mean

Questions you may be asked about this assignment, with short answers. Lines starting with "Follow-up" are the questions an interviewer is likely to ask next. Numbers come from the design document and the tests.

Jump to:

- 30-second pitch
- 1. The problem and the approach
- 2. Data model
- 3. The scheduler and the planner
- 4. Workers and the 5-file limit
- 5. Failures and reliability
- 6. Multiple jobs and fairness
- 7. Technology choices
- 8. Performance and scale
- 9. Testing
- 10. UI and API (bonus)
- 11. Honest limitations
- 12. "What if" questions
- Numbers to remember

30-second pitch

> The user sends F and C. The API splits the job into small tasks and puts them on a queue. A pool of workers takes one task at a time. Each worker returns **sums and a count**, never averages. A scheduler combines those partial results, 5 at a time, as soon as they are ready. The last task divides once. A count check (count = F) tells us the job is really done.

---

1. The problem and the approach

Q: What does the system do?

- It computes the mean of every position across F files of C random numbers.
- The work is split into tasks and run by W workers, 5 files at most per task.

Q: Why sums and a count instead of averages?

- Averages of groups of different sizes give the wrong answer. Example: the mean of [1, 2, 3] is 2 and the mean of [4] is 4. The average of the two is 3, but the true mean is 2.5.
- Sums and counts can be added in any order and any grouping.
- Divide once at the end: total sum / total count.

> **Follow-up: Could you use weighted averages instead?**
>
> - Yes, weighting each average by its count gives the same result.
> - Sums are simpler: no division and no rounding until the very end.

Q: Is this MapReduce?

- It is the same idea, hand-built. Generate tasks are the map step. Combine tasks are the reduce step.
- Each partial result is a (position → sum, count) pair, like a key-value pair.
- I didn't use Hadoop or Spark, because the assignment asks for the distribution logic itself.

> **Follow-up: Where is the shuffle step?**
>
> - There isn't one. Every task works on the same keys: the positions 0 to C-1.
> - A combine just adds two position-by-position lists, so no regrouping is needed.

Q: How many tasks does a job create?

- F / 5 (rounded up) generate tasks, plus about (F − 1) / 4 combine tasks.
- Why (F − 1) / 4: a combine takes up to 5 items and gives back 1, so it removes 4. F items must shrink to 1.
- At F = 99,999 that is about 20,000 + 25,000 = 45,000 tasks.

Q: Why do workers generate the files, not the API?

- It keeps the API fast, and the generation runs in parallel.
- Combining can start while other files are still being generated.

---

2. Data model

Q: What is an item?

- One data file on disk: a raw file (count 1) or a partial result (count n).
- The database row stores only the path, the count and the state. The numbers stay in the `.npy` file.

Q: How are tasks and items related?

- A task produces items (`produced_by`) and a combine task consumes items (`used_by`).
- A GENERATE task has no input items. A COMBINE task takes up to 5.
- The final task's output is the result file, which is stored on the job.

> **Follow-up: Why have an items table and not just keep inputs in the task?**
>
> - The planner needs to ask "which items of this job are waiting?" quickly.
> - A row per item gives a state flag, a unique path, and a link to who produced and used it.

Q: What are the waiting items?

- Items with state `READY`. They wait for a combine task.

- When the planner creates the task, it sets its inputs to `USED` in the same transaction. No other task can take them.

Q: Why store numbers in files and not in the database or the queue?

- Queue messages stay tiny: paths and counts only.
- A file of 999,999 values is about 4 MB. Putting that in a row or a message would slow everything down.
- `.npy` is much smaller than CSV and faster to read.

Q: Why float32 on disk but float64 for the sums?

- float32 halves the file size.
- Adding thousands of values in float32 loses precision. Adding in float64 doesn't.

> **Follow-up: Does float32 make the result less correct?**
>
> - The inputs are float32 by design. The sums are exact float64 sums of them.
> - My check against `numpy.mean` on the same files shows a difference of 0.

---

3. The scheduler and the planner

Q: What does the scheduler do?

- It applies each worker result in one transaction.
- It decides when to create combine tasks, sends backup copies, finds dead workers and publishes pending tasks.

Q: What are the planner rules?

1. 5 or more items waiting: combine 5. Repeat.
2. All files generated, nothing else running, 2 or more waiting: combine them all as the final task.
3. A worker is idle, the queue is empty, 2 or more waiting: combine what is there, so the worker isn't wasted.
4. One item left and everything done: a final task that only divides.

> **Follow-up: Isn't rule 3, a small combine, inefficient?**
>
> - It makes more tasks, but it only fires when a worker is idle and the queue is empty. The alternative is an idle worker.

Q: Do you wait for a whole level to finish before combining?

- No. An item is combined as soon as 5 are ready, from any level.
- A worker never waits for a slow one to finish a "round".

Q: Does the scheduler poll the database or the queue?

- The planner reads the database. The queue holds only tasks that already exist.

- It runs after each result and every 500 ms. The tick catches events with no message: an idle worker, a dead worker, a slow task.

Q: Isn't polling wasteful?

- It is a few small queries per running job, about twice a second.
- The upside is that one timer covers every case and fixes itself if a trigger is missed.
- A cheaper version would run only on change, plus a slow safety timer.

> **Follow-up: How would you make it event-driven?**
>
> - Run the planner after each result, on a worker heartbeat change, and on a slow safety timer (for example every 5 s).
> - The risk is a code path that forgets to trigger it. The tick avoids that.

Q: Why exactly one scheduler?

- It makes every planning decision, so two planners can't race.
- All its state is in the database, so a restart loses nothing.
- The cost: it is a single point of failure for scheduling.

> **Follow-up: How would you run two?**
>
> - Elect a leader with a Postgres advisory lock. The other instance waits as a standby.
> - Or split the jobs between schedulers, for example by job id, so each job has one planner.

Q: Why lock rows with `FOR UPDATE`? Doesn't the database do that?

- It locks rows it changes, not rows it only reads. We read, decide, then write.
- Without the lock, the result handler and the 500 ms tick could plan the same job at once and create two tasks from the same items.
- A cancel could also land between the read and the write.

> **Follow-up: Why not `SERIALIZABLE` isolation instead?**
>
> - The database would abort conflicting transactions, and the code would have to catch the error and retry.
> - A lock is easier to read and to reason about.
>
> **Follow-up: Can the locks cause a deadlock?**
>
> - Code that locks several rows always takes the task first, then the job. Same order everywhere, so no cycle.

Q: How do you know a job is finished?

- The final task is created only when all files exist, nothing else is running, and fewer than 5 items are left.
- The result must have count = F. Then the job is `COMPLETED`.
- If not, it is `FAILED` with a message. A missed or doubled file never turns into a wrong mean.

> **Follow-up: What do you do if count is not F?**
>
> - The job is marked `FAILED` with the message "count mismatch". No result is published.
>
> **Follow-up: Can that happen?**
>
> - Not unless there is a bug. It is a safety net, not an expected case.

Q: How is progress calculated?

- (files generated + merges done) / (2F − 1), capped at 99% until the job completes.
- My first formula showed 0% during generation, so I changed it.

> **Follow-up: Why not just tasks done divided by tasks total?**
>
> - The total isn't known at the start: combine tasks are created as results arrive.
> - The formula uses the work that is known: F generations and F-1 merges.

---

4. Workers and the 5-file limit

Q: How do you enforce "at most 5 files per worker"?

1. The planner never makes a task with more than 5 inputs. A partial counts as one.
2. `prefetch = 1`: a worker holds one task at a time.
3. The worker rejects a task with more than 5 inputs.
4. It loads one input at a time.
5. A 512 MB memory cap per worker process.

> **Follow-up: If the planner already limits it, why check again in the worker?**
>
> - Defence in depth. The limit simulates RAM, so it must hold even if one layer has a bug.
> - The worker check is cheap, and it fails the task with a clear error.
>
> **Follow-up: Does a partial result count as one file or as several?**
>
> - One input. That is why a combine can merge 5 partials that each stand for hundreds of files.

Q: How much memory does a combine need at the maximum size?

- One input is about 4 MB. The running total is about 8 MB (float64).
- Files are read memory-mapped, so they aren't fully loaded.

Q: How do you handle workers of different speeds?

- Workers pull one task when they are free, so fast workers do more tasks without anyone planning it.
- For the demo, a slowdown factor per worker simulates slow machines. The default `1,1,3` makes every third worker 3× slower.

Q: What if one worker is very slow?

- A **backup copy**: if a worker is idle, the queue is empty and a task has run longer than 2× the median (and at least 1 s), I publish one duplicate.

- The first result wins. The other is ignored.
- I tested it with one worker 20× slower (F = 40, C = 999,999). The job took 4.6 to 6.8 s without backup copies and 3.6 to 4.2 s with them, and the result was exact.

> **Follow-up: What if both copies finish?**
>
> - The first result is applied. The second finds the task `DONE` and is ignored.
> - Both write the same file with the same content, so nothing conflicts.
>
> **Follow-up: Doesn't that waste work?**
>
> - Only a little. A backup is sent only when a worker is idle and the queue is empty, and only once per task.

Q: How does the worker know the median?

- It doesn't. The scheduler computes the median of recently finished tasks of the same type.

Q: How do you change the number of workers (W)?

- `PUT /workers` saves the wanted count in the database.
- The worker manager checks every 2 s and starts or stops processes.
- On scale-down the worker gets `SIGTERM`, finishes its current task, then exits.

Q: Why does the worker block SIGTERM during a task?

- A signal in the middle of a file write or a network call could break the task.
- The signal waits until the task is acknowledged, then the worker exits cleanly.
- It is a "do not disturb" sign, not a transaction. Nothing is rolled back.

> **Follow-up: What about `SIGKILL` or a power cut?**
>
> - `SIGKILL` can't be blocked. The task is lost for that worker, and the broker redelivers it.
> - That is the failure path we already handle.

Q: Why doesn't the API start or stop containers itself?

- That needs access to Docker, which is a security risk.
- A manager process plus a database setting is simpler and safer.

---

5. Failures and reliability

Q: What if a worker crashes mid-task?

- Its task was never acknowledged, so RabbitMQ gives it to another worker.
- The manager restarts the crashed process.
- I tested with `kill -9` during a job: the job completed with an exact result.

Q: What if a result arrives twice?

- A result for a task that is already `DONE` is ignored.

- `items.path` is unique and the insert uses `ON CONFLICT DO NOTHING`.
- Tasks write to a fixed path, and the random seed comes from the job id and file index, so a retry writes identical data.

Q: Is delivery exactly-once?

- Not strictly. RabbitMQ gives at-least-once delivery, so a message can arrive twice.
- The system makes the *effect* exactly-once by handling every duplicate safely (idempotent handling).

> **Follow-up: If you say it is effectively exactly-once, how do you handle it?**
>
> - **Ack after commit.** If the scheduler crashes before the ack, the result comes again.
> - **Task already `DONE`?** The repeat is ignored.
> - **`items.path` is unique,** and the insert uses `ON CONFLICT DO NOTHING`, so an item is never added twice.
> - **Same output on a retry.** The seed comes from the job id and file index, and the path is fixed.
> - **Final check:** count must equal F, so a lost or doubled file can't become a wrong mean.
>
> **Follow-up: Can a duplicate ever change the result?**
>
> - No. A duplicate is ignored before it touches any item or counter.
> - It is covered by an integration test: the same result sent twice gives one set of items.
>
> **Follow-up: What if the scheduler commits and crashes before the ack?**
>
> - RabbitMQ redelivers the message. The task is already `DONE`, so it is logged as a duplicate and dropped.

Q: When do you acknowledge a message?

- The scheduler acks only after the database transaction commits. A crash before that means a redelivery.
- The worker acks only after its result is published and confirmed by the broker.

> **Follow-up: What if the worker dies after publishing its result but before the ack?**
>
> - The task is redelivered and computed again. The first result is already applied, so the second is dropped as a duplicate.

Q: How do you avoid half-written files?

- Files are written under a temporary name and then renamed. A rename is atomic.
- A reader sees either nothing or a complete file.

> **Follow-up: What if two workers write the same file at once (a backup copy)?**
>
> - Each writes to its own temporary name (it includes the process id), then renames it over the final path.
> - The rename is atomic, and the content is identical, so either order gives the same file.

Q: What if the same task fails again and again?

- It gets 3 attempts, then the task and the job are marked `FAILED` with the error.

Q: How do you detect a dead worker?

- Workers send a heartbeat every 2 s. No heartbeat for 10 s marks the worker `DEAD`.
- Its task is redelivered by the broker.

> **Follow-up: How does its task get redelivered?**
>
> - The task was never acknowledged. When the worker's connection closes, RabbitMQ puts the message back on the queue.
>
> **Follow-up: What if the worker is frozen, not crashed?**
>
> - The connection stays open, so RabbitMQ waits for its own heartbeat to time out. That can take minutes.
> - It is a known limit. A shorter broker heartbeat, or re-publishing the task from the scheduler, would fix it.

Q: What if the scheduler crashes?

- An unfinished transaction rolls back, and the unacknowledged result is redelivered.
- On restart it continues from the database state.

Q: What if RabbitMQ restarts?

- Queues and messages are durable. The scheduler reconnects and picks the consumer back up.
- I restarted it in the middle of a 2,000-file job. The scheduler reconnected and the job finished with an exact result.

Q: What if a bad message appears on the results queue?

- It is checked, logged and dropped. Retrying a bad message would fail the same way forever.
- A real failure, like a database error, is requeued.

Q: What happens when a job is cancelled?

- Tasks not yet published are failed at once.
- Running tasks finish, but their results are discarded. Workers skip tasks of a cancelled job.

> **Follow-up: What about the files already written?**
>
> - They stay on disk. There is no cleanup yet. It would be a small job that deletes a cancelled or old job's folder.

---

6. Multiple jobs and fairness

Q: How do many jobs run at once?

- All tasks of all jobs share one worker pool. Each task carries its job id.

Q: How do you stop a large job from blocking a small one?

- Only about 2 × W tasks wait in the queue. The queue is refilled in turns: the first task of every job, then the second of every job.
- Measured: a job with F = 10, submitted after one with F = 3,000, finished in 3 s while the big job took 37 s.

> **Follow-up: Can a job starve?**
>
> - No. Every job gets one task per round, however many tasks the others have.
>
> **Follow-up: How would you add job priorities?**
>
> - Weight the rounds: a high-priority job gets 2 or 3 tasks per round instead of 1.

Q: Why not put all tasks on the queue at once?

- A large job would fill the queue and every later job would wait behind it.

---

7. Technology choices

Q: Why RabbitMQ?

- One task at a time per worker (`prefetch`), redelivery of unacknowledged tasks, durable queues, and confirms when publishing.
- Redis would need redelivery written by hand. SQS needs AWS. A database table needs polling.

> **Follow-up: What if RabbitMQ loses a message?**
>
> - Queues are durable, messages are persistent, and publishing waits for the broker's confirm.
> - A message is deleted only after it is acknowledged.
>
> **Follow-up: Why not Kafka?**
>
> - Kafka is built for ordered logs and replay. I need one task per worker with an acknowledgement for each task.

Q: Why PostgreSQL?

- Transactions, row locks and `SKIP LOCKED`, which the planner and dispatcher depend on.

> **Follow-up: Why not use the database as the queue too?**
>
> - It can work with `SKIP LOCKED`, but workers would have to poll, and redelivery would be code I write myself.
> - RabbitMQ gives both out of the box.

Q: Why Python for workers and TypeScript for the API?

- The assignment asks for a Node/Express/TypeScript API and Python workers.
- numpy makes the array maths short and fast.

Q: Why Docker Compose?

- One command starts everything: Postgres, RabbitMQ, API, scheduler and workers.

- The scheduler uses the same image as the API with a different entry point.

Q: Why no ORM?

- The queries are specific (`SKIP LOCKED`, window functions, `ON CONFLICT`). Plain SQL is clearer here.
- All SQL lives in the repository layer, nowhere else.

---

8. Performance and scale

Q: Can it handle F = 99,999 and C = 999,999?

- By design, yes. I never ran that size: the raw data would be about 400 GB.
- The largest runs were F = 4,000 with C = 300,000, and C = 999,999 with F = 40.

Q: What would break first at maximum size?

1. The planner reads all of a job's waiting items on each pass.
2. The single scheduler.
3. Disk space for the shared folder.

> **Follow-up: How would you fix the planner?**
>
> - Read only a few waiting items per pass (`LIMIT 5`), or keep a count so a job with fewer than 2 waiting is skipped.

Q: How would you scale it further?

- Add leader election or an advisory lock for a standby scheduler.
- Move file storage to S3 or a similar object store.

Q: How good is your scheduling compared to the best possible?

- A simulation (`tools/simulate.py`) with 8 workers of different speeds compared four strategies.
- Each number is how many times longer the job took than the best possible time. Lower is better, and 1.00× is perfect.
- **Split the files evenly at the start:** 1.70× to 2.98×.
- **Combine in fixed rounds:** 1.01× to 2.06×.
- **Combine as soon as items are ready:** 1.01× to 3.43×.
- **Same, plus backup copies (mine):** 1.00× to 2.06×. It is within 2% of perfect from F = 1,000 up.
- Small jobs (F = 20) are poor for every strategy, because 20 files make too few tasks to keep 8 workers busy.

Q: How deep is the combining tree?

- About 7 to 8 levels at F = 99,999, since each combine takes 5 inputs. It is eager, so levels overlap.

---

9. Testing

Q: How did you test it?

- 11 unit tests of the maths.
- 35 integration checks on a real PostgreSQL: every planner rule, result handling, validation and publish order.
- 920 random jobs through a model of the planner rules (`tools/planner_check.py`).
- End-to-end runs: exact match with `numpy.mean` (difference 0), worker `kill -9`, several jobs at once, cancel, scaling, a slow worker, a RabbitMQ restart and a scheduler restart.
- 21 click-through checks of the UI in a simulated browser.

> **Follow-up: How did you test crash recovery?**
>
> - I ran `kill -9` on a worker during a job (`tools/kill-worker.sh`). The task was redelivered and the result matched numpy exactly.
> - I also restarted RabbitMQ in the middle of a job. It finished correctly.

Q: How do you verify a result is correct?

- `python tools/verify.py <jobId>` recomputes the mean from the raw files and compares.
- `tools/show_job.py` prints the raw values next to the mean, so you can check by eye.

Q: What did you not test?

- F = 99,999 with C = 999,999.
- The UI buttons in a real browser. Pages were checked in headless Edge and a simulated browser (jsdom), and the API calls behind each button were tested directly.

---

10. UI and API (bonus)

Q: What does the UI offer?

- Overview, Jobs (filter and search), Job detail (progress, task counts, result preview, download, cancel, its own log), Workers (scale W) and Activity.
- Plain HTML, CSS and JavaScript, no build step. It refreshes every second.

Q: Why polling and not WebSockets?

- It is simple and works through any proxy. A refresh each second is fine for this size.
- Server-sent events would be the next step.

Q: What does the API check?

- 1 < F < 100,000 and 1 < C < 1,000,000. Ids must be valid UUIDs, otherwise `404`.
- All SQL uses query parameters, and the UI inserts text with `textContent`.

---

11. Honest limitations

Q: What are the weak points?

- Single scheduler.

- No authentication (out of scope). The database login is for local use only.
- Dead worker rows stay in the table.
- Results aren't cleaned up over time.

Q: What known issues did a code review find that are still open?

- Several jobs can each create a small combine in the same tick for one idle worker, because the tick reuses one fleet snapshot.
- When a backup copy runs, the "waiting in queue" count can read one too low.
- `backup_sent` isn't reset when a task is retried.
- The API and the scheduler can both publish and briefly overfill the queue.
- A frozen worker (not crashed) keeps its task until the broker heartbeat times out.
- A request body that is not valid JSON returns `500` instead of `400`.
- None of them gives a wrong result. They affect efficiency, recovery time or an error code.

Q: What would you do differently with more time?

- Incremental planning for huge jobs.
- A dead-letter queue and metrics.
- Object storage instead of a shared folder.
- Real-time updates with server-sent events.
- Authentication and cleanup of old jobs.

---

12. "What if" questions

Q: What if F is not a multiple of 5?

- The last generate task has fewer files (for example F = 12 gives groups of 5, 5 and 2).

Q: What if there is only one worker?

- It works, just slower. The queue holds at least 2 tasks, and the worker takes them one by one.

Q: What if W is set to 0?

- Nothing runs. Tasks wait in the database and the queue until workers exist.

Q: What if two users submit identical jobs?

- They are two independent jobs. Their files are in separate folders named by job id.

Q: What if the disk fills up?

- A write fails, the task reports `FAILED`, and it is retried up to 3 times. Then the job fails with the error.

Q: What if the database is down for a moment?

- Failed results are requeued. The scheduler's tick logs the error and tries again 500 ms later.
- The API returns an error until the database is back.

Q: How would you add a different job type, such as a median or a sum?

- A sum fits the same design: skip the division.

- A median can't be combined from partial results. It needs a different approach.

Numbers to remember

- **Limits:** 1 < F < 100,000 and 1 < C < 1,000,000
- **Inputs per task:** at most 5
- **Tasks at F = 99,999:** about 20,000 generate + 25,000 combine
- **Queue depth:** about 2 × W
- **Scheduler tick:** 500 ms
- **Heartbeat / dead after:** 2 s / 10 s
- **Attempts per task:** 3
- **Backup copy:** running longer than 2× the median, and at least 1 s
- **Memory cap:** 512 MB per worker
- **Raw data at maximum size:** about 400 GB
- **Tests:** 11 unit + 35 integration